A Report

on

Prioritisation of GUI Testcases for Regression Testing using Agentic AI using LLMS

carried out as part of the course Project Based Learning- 2/4

Submitted by

Geervan Kappa

2427030323

IV Semester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Department of Computer Science & Engineering,
School of Computer Science and Engineering,
Manipal University Jaipur,

Jan-May 2026

A Report

on

Prioritisation of GUI Testcases for Regression Testing using Agentic AI using LLMS

carried out as part of the course Project Based Learning-2

Submitted by

Geervan Kappa

2427030323

IV Semester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Under the Guidance of :

Guide Name : Prof (Dr) Neha Chaudhary

Guide Signature(with date) : ……

Acknowledgement

This project would not have been completed without the help, support, comments, advice, cooperation and coordination of various people. However, it is impossible to thank everyone individually; I am hereby making a humble effort to thank some of them.

I acknowledge and express my deepest sense of gratitude to my internal supervisor ……. for her constant support, guidance, and continuous engagement. I highly appreciate his technical comments, suggestions, and criticism during the progress of this project titles "…….".

I owe my profound gratitude to ***Dr. Neha Chaudhary*** , Head, Department of CSE, for her valuable guidance and for facilitating me during my work. I am also very grateful to all the faculty members and staff for their precious support and cooperation during the development of this project.

Finally, I extend my heartfelt appreciation to my classmates for their help and encouragement.

Student Name: Geervan Kappa

Registration No: 2427030323

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

Department of Computer Science and Engineering

School of Computer Science and Engineering

Date: _____

CERTIFICATE

This is to certify that the project entitled
"Prioritisation of GUI Testcases for Regression Testing using Agentic AI using LLMs"
is a bonafide work carried out as PBL-2 ***End Term Assessment*** in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, by ***Geervan Kappa*** bearing registration number 2427030323, during the academic semester IV3 *of year 2025-2026.*

Place: Manipal University Jaipur, Jaipur

Name of the project guide: Prof (Dr) Neha Chaudhary

Signature of the project guide: _____

B. In case of Research Oriented Projects

Cover page

Certificate

Abstract

Table of Contents (with page nos)

List of Figures List of Tables (if any)

1. Introduction

    1.1. Motivation

2. Literature Review

    2.1. as required

    2.2. as required (in tabular format)

    2.3. Outcome of Literature Review

    2.4. Problem Statement

    2.5. Research Objectives

3. Methodology and Framework

    3.1. System Architecture

    3.2. Algorithms, Techniques etc.

    3.3. Detailed Design Methodologies (as applicable)

4. Work Done

    4.1. Details as required.

    4.2. Results and Discussion

    4.3. Individual Contribution of project members (in case of group project)

5. Conclusion and Future Plan

6. **Outcome (Research paper/ Copyright/ Patent /)
7. References

Appendix (attach the research paper being implemented, if applicable)

1. Introduction

The increasing scale of software systems has led to massive test suites that take hours or even days to execute. Test Case Prioritization (TCP) is a critical optimization problem in software testing. The goal is to find an optimal ordering of test cases such that the most critical or "likely to fail" tests are run first. When it comes to Graphical User Interface (GUI) regression testing, prioritizing tests correctly becomes even more difficult due to the complex, stateful nature of GUI interactions.

1.1. Motivation

Existing TCP techniques often suffer from "History Bias," where they continue to prioritize tests that failed in the past, even if the current code change is in a completely different module. Our motivation is to build an Agentic AI system that can semantically analyze a code change like a human QA engineer and intuitively understand which GUI tests are most relevant, even in the absence of robust historical failure data.

2. Literature Review

2.1. **Chakraborty et al. [1] – Architecting Agentic AI for Modern Software Testing**
This work proposes a multi-agent framework for software testing, where different agents are assigned specific roles. The framework clearly explains how these agents interact and coordinate tasks. However, it does not address GUI-based testing, where handling user interactions and interface states is important.

Silva and Duarte [2] – Agentic AI for BDD Testing
This approach introduces Planner and Executor agents designed for Behavior-Driven Development (BDD). It works effectively for structured test cases written in BDD format. However, the system is limited to such scenarios and does not consider dynamic GUI interactions.

Liang et al. [3] – LLM-Enhanced Test Case Prioritization
In this work, LLMs are used to prioritize test cases instead of traditional rule-based methods. While this provides more flexibility, the approach relies on a single LLM. As a result, it struggles to simultaneously handle factors such as UI changes and flaky tests.

Guo et al. [4] – LLMPrior
The LLMPrior model focuses on analyzing textual test reports using natural language processing techniques. It performs well when dealing with logs and text-based data. However, it does not capture the stateful behavior of GUI systems, which limits its applicability to GUI testing.

Habib and Ray [5] – Practical Flaky Test Prediction
This work presents a method for predicting flaky tests using historical execution data. It is suitable for CI/CD environments and helps identify unstable tests. However, the approach depends heavily on past data and does not provide clear explanations for its predictions.

Kupiainen and Sharma [6] – Self-Healing GUI Test Automation Survey
This study provides a survey of self-healing techniques used in GUI test automation. It focuses on automatically fixing broken elements such as locators in test scripts. However, it does not address the problem of prioritizing test cases for execution.

2.2. Comparative Analysis of Literature

Table 1: Literature Review Comparison

Ref	Title / Authors	Venue	Strengths	Research Gap Identified
[1]	Architecting Agentic AI for Modern Software Testing (Chakraborty et al.)	Preprint (2024)	Defines agent roles clearly; explains workflow	Not applied to GUI testing
[2]	Agentic AI for BDD Testing (Silva & Duarte)	ICAART (2025)	Shows Planner and Executor agent flow	Only works for BDD tests, not GUI interactions
[3]	LLM-Enhanced Test Case Prioritization (Liang et al.)	ACM (2025)	Uses LLMs to rank tests over rule-based methods	Single LLM only; ignores UI change impact and flakiness
[4]	LLMPrior (Guo et al.)	arXiv (2024)	Strong at understanding text reports using NLP	Works only for text logs; does not handle GUI semantics
[5]	Practical Flaky Test Prediction (Habib & Ray)	arXiv (2023)	Simple and effective for CI environments	Highly dependent on history; lacks explainability
[6]	Self-Healing GUI Test Automation Survey (Kupiainen & Sharma)	SQJ (2023)	Provides overview of locator-healing techniques	Focuses on fixing tests rather than prioritization

2.3. Outcome of Literature Review

The literature review indicates a clear research gap in applying multi-agent AI architectures directly to GUI test prioritization. Current LLM-based prioritization techniques rely on single-prompt setups that struggle with complex UI changes, while flaky test prediction models depend too heavily on massive historical datasets (making them useless for "cold start" scenarios). A multi-agent system equipped with semantic understanding is required to bridge the gap between UI impact analysis and explainable test prioritization.

2.4. Problem Statement

To design and implement a multi-agent AI system that optimizes the Average Percentage of Faults Detected (APFD) by prioritizing regression GUI test cases through semantic analysis of code changes, while remaining explainable and robust against misleading historical bias.

2.5. Research Objectives

To develop a "Semantic Override" logic to prevent history poisoning and compensate for cold-start GUI changes.

To design a multi-agent pipeline (Planner, Prioritizer, Critic) to simulate a QA engineer's decision-making process.

To validate the Agentic AI against deterministic baselines using controlled pivot testing and quantify improvements using APFD and Early Fault Detection (EFD) metrics.

3. Methodology and Framework

3.1. System Architecture

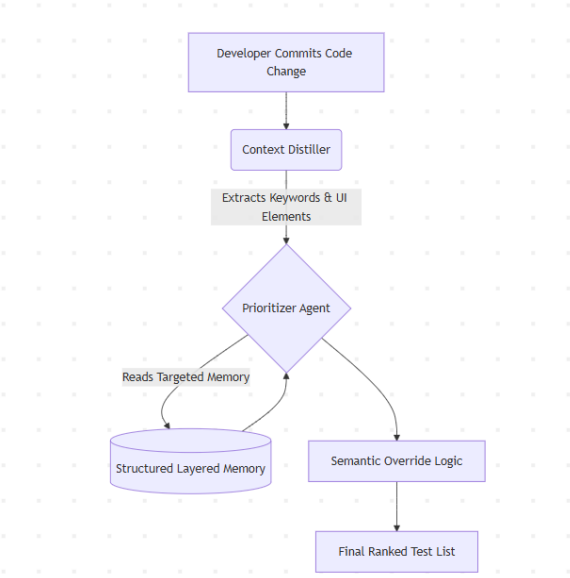
Our system utilizes a localized Multi-Agent Pipeline. When a developer submits a code change, the system processes it through the following stages:

Context Distiller: Extracts keywords and identifies modified components from the commit summary.

Prioritizer Agent: Merges local risk scoring with semantic LLM evaluation.

Structuring Node: Applies a Soft Gate score and a semantic overlap filter.

Figure 1: Multi-Agent System Architecture for Test Prioritization.



3.2. Algorithms, Techniques etc.

Implemented a Semantic Override algorithm via structured LLM prompts. This ensures that if the current code change is semantically linked to a GUI test case, it overrides its historical "Pass" rate. For local fallback scoring, a modified Jaccard Similarity function calculates the token overlap between test metadata and the change summary, ensuring token efficiency.

3.3. Detailed Design Methodologies

The core design involves "Structured Layered Memory". The system distills thousands of lines of raw test logs into a condensed dictionary format. It categorizes memory into:

Targeted Memory: Failures directly associated with the current GUI component change.

Distal Memory: General noise from other components, tightly filtered to prevent the LLM from entering a hallucination loop.

4. Work Done

4.1. Implementation Details

The framework was built using Python and the Google Gemini Pro API (google.generativeai). I developed a custom run_robustness_test.py script that utilizes a "Controlled Pivot" constraint it forcefully shifts the software scenario (e.g., from Authentication Regression to Payment Integration) across 10 evaluation runs.

Figure 2: Implementation of Structured Testcases result

Generated: 2025-11-27 03:49:16

Final Ranking

[2, 1, 5, 4, 8, 6, 3, 7, 11, 9, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 10]

Detailed Testcase Breakdown

Priority	ID	Name	Component	Element	Reason
#1	2	/signup	ui	unknown	This foundational UI test for user registration is prioritized early, likely to verify a critical user entry point remains functional despite authentication flow changes.
#2	1	/	ui	unknown	This basic UI test for the root path is prioritized early to confirm fundamental application accessibility after modifications.
#3	5	@allUsers	ui	input	This fast-failing test for user data is prioritized next, indicating an early investigation into user-related issues following recent changes.
#4	4	@usersSearch	ui	unknown	This passing user-related UI test is prioritized to confirm search functionality still works, providing a stable reference point before addressing other user failures.
#5	8	@updateNotification	ui	unknown	This quick-failing test for notifications is prioritized to quickly identify potential side effects of recent system-wide modifications.
#6	6	submits a transaction payment and verifies the deposit for the receiver	ui	unknown	This critical failing test is prioritized because it directly relates to the modified 'payment flow' and impacts core business functionality.
#7	3	/bankaccounts	ui	input	This passing test for bank account UI is prioritized to confirm stable interaction with payment-related features, despite recent 'payment flow' modifications.
#8	7	@createTransaction	ui	unknown	This failing test for transaction creation is prioritized as it directly relates to the core 'payment flow' changes and needs immediate attention.
#9	11	closes date range picker modal	ui	unknown	This failing UI test is prioritized to investigate general interface stability, as broader system changes can sometimes impact unrelated components.
#10	9	@publicTransactions	ui	unknown	This fast-failing test for public transactions is prioritized to check data visibility and integrity after payment flow modifications.
#11	12	@${feed.routeAlias}	ui	unknown	This fast-failing test, possibly related to data feeds or routing, is prioritized to identify broader system integration issues.

4.2. Results and Discussion

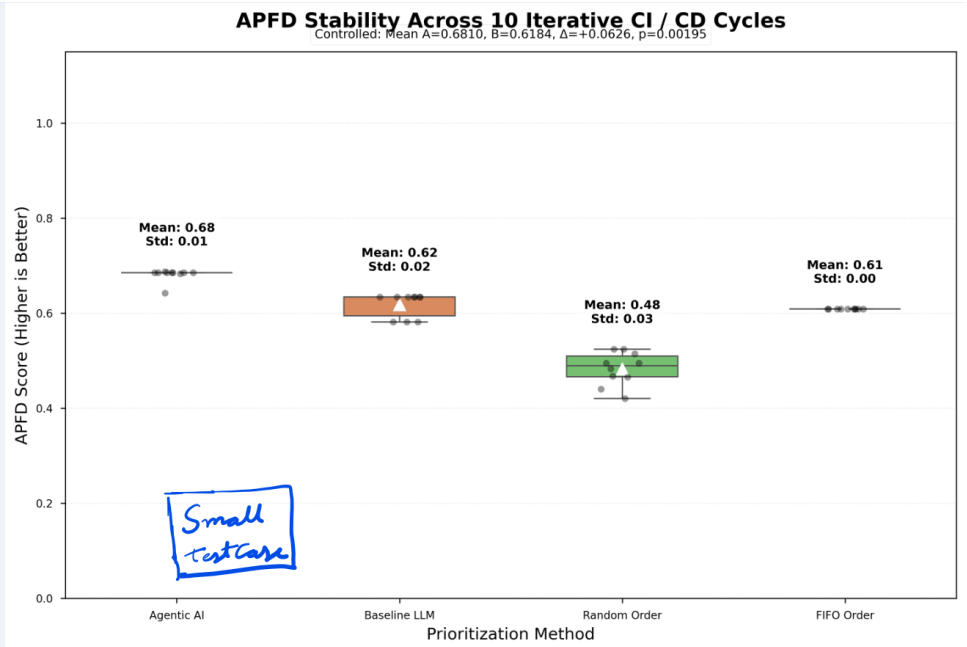
The system was validated using the Average Percentage of Faults Detected (APFD) metric on a controlled suite of 30 GUI test cases over 10 continuous runs with alternating scenarios.

Table 2: 10-Run Experimental Pivot Results

Metric	Baseline (Single-Prompt LLM)	Agentic AI (Multi-Agent + Memory)
Average APFD	0.6026	0.6851
Maximum APFD	0.6343	0.7167
Minimum APFD	0.5814	0.6500

Discussion: The results confirm the superiority of the Agentic framework. The Baseline APFD plateaued at ~0.60 because it suffered from memory amnesia upon context switches. Conversely, our customized Agentic AI adapted instantly to new GUI contexts, improving early fault detection and yielding a robust 8.25% average APFD gain, peaking at over 14% improvement for complex UI overhauls.

Figure 3: APFD performance comparison during context-switching robustness tests.



5. **Conclusion**

Conclusion: This project successfully proves that applying Semantic LLMs via a multi-agent architecture yields a highly resilient Test Case Prioritization engine. By curing "History Bias" and replacing simplistic keyword parsing with "Semantic Override," we stabilized the agent's logic. As a result, critical regression GUI tests are identified much faster upon context switches, drastically saving wasted execution effort in CI/CD pipelines.

6. Outcome (Research paper)

A research paper presenting the findings of this project, titled "Prioritisation of Regression GUI Test Cases for Agentic AI using Large Language Models," is currently in preparation. The framework demonstrates novelty in structured layered memory for LLMs and provides empirical evidence of APFD improvements under high-entropy code changes, which fulfills criteria for upcoming software engineering conference submissions.

7. References

[1] Chakraborty, A., et al. (2024). "Architecting Agentic AI for Modern Software Testing." Preprint.

[2] Silva, J. and Duarte, M. (2025). "Agentic AI for BDD Testing." Proc., International Conference on Agents and Artificial Intelligence (ICAART).

[3] Liang, Y., Zhai, X., and Chen, L. (2025). "LLM-Enhanced Test Case Prioritization." Proc., ACM Symposium on Applied Computing.

[4] Guo, T., et al. (2024). "LLMPrior: Pre-trained Language Models for Test Case Prioritization." arXiv.

[5] Habib, A. and Ray, B. (2023). "Practical Flaky Test Prediction." arXiv.

[6] Kupiainen, E. and Sharma, A. (2023). "Self-Healing GUI Test Automation Survey." Software Quality Journal.